Notes on Grammatical Analysis

Sinan Olsson-Pasic

January 9, 2026
(revised May 26, 2026)

The following are a collection of notes that I'm documenting during my development and learning process of my grammatical analysis engine.

1 What is a language?

A language to a computer scientist is a possibly infinite set of sentences, each made up of "tokens" (words), and each token made up of "symbols" (letters). Or in other words, a language is a subset Σ^* for some alphabet Σ. Which "sentences" belong to a "language" and which "tokens" make up those "sentences" is explained by the definition of a generative grammar for the language.

A generative grammar is a 4-tuple (V_N, V_T, R, S) such that:

1. V_N and V_T are finite sets of symbols.
2. $V_N \cap V_T = \emptyset$
3. R is the set of pairs (P, Q) such that:
   (a) $P \in (V_N \cup V_T)^{*+}$
   (b) $Q \in (V_N \cup V_T)^*$
4. $S \in V_N$

Where V_N is the set of non-terminals, V_T is the set of terminals, R is the set of rules, and S is the start symbol. $\emptyset$ denotes the empty set, meaning the intersection of the terminals and nonterminals returns no element.

A language L bound by a grammar G therefore is defined as:

$$L(G) = \{\, w \in V_t^* \mid S \Rightarrow^* w \,\} \tag{1}$$

All of this is to say that a generative grammar gives us the specification of how sentences for a language is constructed rather than listing the valid sentences. This allows us to describe infinite languages using finite information.

1.1 On the derivation relation

The arrow $\Rightarrow^*$ hides the mechanism of generation, so it's worth unfolding. A rule $(P, Q) \in R$ licenses one rewriting step: a string $\alpha P \beta$, with the pattern P sitting in some context α, β, becomes $\alpha Q \beta$. Call this one-step relation $\Rightarrow$.

Definition 1.1 (Derivation)**.** $\Rightarrow \subseteq (V_N \cup V_T)^* \times (V_N \cup V_T)^*$ is the relation with $\alpha P \beta \Rightarrow \alpha Q \beta$ whenever $(P, Q) \in R$. Its reflexive–transitive closure is $\Rightarrow^*$: $\alpha \Rightarrow^* \gamma$ holds iff $\alpha = \gamma$ or there is a finite chain $\alpha = \gamma_0 \Rightarrow \gamma_1 \Rightarrow \cdots \Rightarrow \gamma_k = \gamma$.

So $\Rightarrow^*$ is $\Rightarrow$ applied any finite number of times, and $L(G)$ collects exactly the terminal strings reachable from S this way. Membership of w in $L(G)$ is an existential statement: it asserts that *some* derivation exists. The same w may have many derivations, or none. A derivation is a proof of membership; its absence is the stronger claim that no proof exists.

1.2 A note on infinite languages and descriptions

Is it possible to give a description for every language? The answer is no. The reason is a counting argument.

Theorem 1.1 (Almost every language is indescribable)**.** *Over a finite alphabet Σ there are uncountably many languages but only countably many grammars. So the languages that admit any finite description are a vanishing fraction of all languages.*

Proof sketch. Σ^* is countable: list strings by length, then alphabetically, for a bijection with $\mathbb{N}$, so $|\Sigma^*| = \aleph_0$. A language is any subset, so the languages form the power set, and $|\mathcal{P}(\Sigma^*)| = 2^{\aleph_0} > \aleph_0$ by Cantor's theorem — uncountably many. (Diagonally: any list $L_0, L_1, \ldots$ misses $D = \{w_i \mid w_i \notin L_i\}$, which disagrees with each L_i on the i-th string w_i.) A grammar is a finite object — finite V_N, V_T, R — hence writable as a finite string, and the finite strings over a fixed alphabet are countable. Countably many descriptions cannot name uncountably many languages. □

The same bound applies to Turing machines, regular expressions, or descriptions of any finitary kind. So the grammar hierarchy below sits entirely inside the countable sliver of describable languages. The question is never whether an arbitrary language can be described, but how much descriptive power natural languages demand, and what that power costs.

2 The Chomsky hierarchy

The 4-tuple above places no restriction on the shape of a rule: any nonempty P may rewrite to any Q. Restricting that shape carves out a sequence of grammar classes, each weaker than the last and each cheaper to decide.

Definition 2.1 (The four grammar types)**.** A grammar (V_N, V_T, R, S) is:

1. **Type 0 (recursively enumerable).** No restriction: any (P, Q) with $P \in (V_N \cup V_T)^{*+}$.
2. **Type 1 (context-sensitive).** Every rule is non-contracting, $|P| \leq |Q|$; equivalently of the form $\alpha A \beta \to \alpha \gamma \beta$ with $A \in V_N$, $\gamma \neq \varepsilon$.
3. **Type 2 (context-free).** Every left-hand side is a single nonterminal: $A \to Q$.
4. **Type 3 (regular).** Every rule is right-linear: $A \to aB$ or $A \to a$.

The names are literal. Context-free: a nonterminal rewrites regardless of its neighbors. Context-sensitive: the frame $\alpha \_ \beta$ must be present for the rewrite to fire. The content is not the rule syntax but the machine each class corresponds to.

Each grammar class is generated exactly by machines with a fixed memory budget. No memory tracks only a bounded amount of information; one stack matches nested brackets but cannot

Type	Class	Recognizer	Memory
0	recursively enumerable	Turing machine	unbounded tape
1	context-sensitive	linear-bounded automaton	tape linear in input
2	context-free	pushdown automaton	a single stack
3	regular	finite automaton	none

Table 1: Grammar power tracks memory.

compare three counts at once; unbounded tape recognizes anything recognizable. The classes nest strictly:

$$\mathsf{Reg} \subsetneq \mathsf{CF} \subsetneq \mathsf{CS} \subsetneq \mathsf{RE},$$

with $\{a^n b^n\} \in \mathsf{CF} \setminus \mathsf{Reg}$ and $\{a^n b^n c^n\} \in \mathsf{CS} \setminus \mathsf{CF}$. The containments are immediate from the definitions; the strictness is what the pumping lemmas of Section 4 establish. These two sets are the abstract shapes that natural-language dependencies turn out to take.

3 The membership problem

The question a grammar checker actually asks is membership: given w, is $w \in L(G)$? The hierarchy answers it at every rung, and the answers set the budget for any checker.

Theorem 3.1 (Cost of membership)**.** *For fixed G and input length n: regular is $O(n)$; context-free is $O(n^3)$; context-sensitive is PSPACE-complete; type 0 is undecidable.*

Proof sketches. Regular. A DFA reads each symbol once; acceptance is a final table lookup.

Context-free. CKY (in Chomsky normal form, $A \to BC$ or $A \to a$) fills a triangular table $T[i, j]$ of all nonterminals deriving the length-j substring at position i: $O(n^2)$ cells, each tried against $O(n)$ split points, so $O(n^3|G|)$. Then $w \in L(G)$ iff $S \in T[1, n]$. The table also records every parse, not just the verdict. (Valiant pushes the constant toward matrix-multiplication time, $\approx O(n^{2.37})$.)

Context-sensitive. A linear-bounded automaton uses tape linear in n, so each configuration fits in polynomial space; configuration reachability is in PSPACE, with a matching reduction for completeness.

Type 0. A derivation can simulate an arbitrary Turing machine, so deciding $S \Rightarrow^* w$ would decide halting. □

A fully general grammar is therefore not merely expensive but unanswerable. A workable checker has to sit at or near context-free: expressive enough for real structure, cheap enough to decide in polynomial time. One distinction is needed before going further.

Definition 3.1 (Weak vs. strong generative capacity)**.** The *weak* capacity of G is the string set $L(G)$; the *strong* capacity is the set of parse trees it assigns. Grammars are weakly equivalent when they generate the same strings, even with different trees.

Deciding "grammatical?" is a weak-capacity question. Locating and naming an error is a strong-capacity one: pointing at "the verb phrase" needs the right tree, not just the right yes/no.

4 Where natural language sits

Two results fix the position of natural language. Both are pumping arguments, so state the tools first.

4.1 Not regular

Lemma 4.1 (Regular pumping)**.** *If L is regular, some $p \geq 1$ has: every $s \in L$ with $|s| \geq p$ factors as $s = xyz$, $|xy| \leq p$, $|y| \geq 1$, and $xy^i z \in L$ for all $i \geq 0$.*

Proof sketch. Let p be the number of DFA states. A string of length $\geq p$ revisits some state (pigeonhole); the loop y between the two visits can be traversed any number of times while staying on an accepting run. □

Proposition 4.2. *$\{a^n b^n\}$ is not regular.*

Proof. With $s = a^p b^p$ and $|xy| \leq p$, the block y is all a's, so xy^2z has more a's than b's and leaves the language. □

Center embedding gives this shape in natural language: "the rat [the cat [the dog chased] killed] ate" nests matching dependencies that count like $a^n b^n$. A finite-state device has no stack to count nesting depth, so natural language needs at least a stack — at least context-free (Chomsky, 1956).

4.2 Not context-free

Lemma 4.3 (Context-free pumping)**.** *If L is context-free, some $p \geq 1$ has: every $z \in L$ with $|z| \geq p$ factors as $z = uvwxy$, $|vwx| \leq p$, $|vx| \geq 1$, and $uv^i wx^i y \in L$ for all $i \geq 0$.*

Proof sketch. A long enough z forces a root-to-leaf path in its parse tree longer than $|V_N|$, so some nonterminal repeats on that path. The subtree between the two occurrences can be excised or duplicated, pumping the frontier pieces v and x together; capping the lower occurrence's height bounds $|vwx|$. □

Proposition 4.4. *$\{a^n b^n c^n\}$ and the copy language $\{ww\}$ are not context-free.*

Proof. For $z = a^p b^p c^p$, the window vwx spans at most two of the three blocks, so pumping changes at most two counts and breaks the equality. The same straddling argument breaks $\{ww\}$, which is the abstract form of a *crossing* rather than nesting dependency. □

Whether natural language stays context-free was open for decades and settled by Shieber (1985), using cross-serial dependencies in Swiss German subordinate clauses: a block of case-marked noun phrases followed by a block of verbs, where each verb must match the correspondingly-placed noun phrase in case — so the dependencies cross.

Theorem 4.5 (Shieber 1985)**.** *Some natural language has a string set that is not context-free.*

Proof architecture. Context-free languages are closed under intersection with regular languages and under homomorphism. For Swiss German D, a suitable regular R and homomorphism h give

$$h(D \cap R) = \{\, a^m\, b^n\, c^m\, d^n \mid m, n \geq 0 \,\},$$

where a, b are the two case classes of noun phrase and c, d the verbs that govern them; crossing forces the interleaved matched counts. Pumping shows this set is not context-free, and since CF is closed under the operations used, D cannot be context-free either. □

So natural language is strictly above context-free, but only just. Joshi (1985) named the target class.

Definition 4.1 (Mildly context-sensitive)**.** A language class is mildly context-sensitive if it contains the context-free languages, captures limited cross-serial dependencies, has the constant-growth property (the length set $\{|w| : w \in L\}$ is semilinear, ruling out $\{a^{2^n}\}$), and is parsable in polynomial time.

Theorem 4.6 (Vijay-Shanker & Weir 1994)**.** *Tree-Adjoining Grammar, Combinatory Categorial Grammar, Linear Indexed Grammar, and Head Grammar are weakly equivalent — one class, properly above context-free, inside the mildly context-sensitive region.*

Four independently motivated formalisms generating one class is good evidence the class is natural. The cost shows in parsing: TAG recognition is $O(n^6)$, polynomial but visibly dearer than $O(n^3)$. In practice a context-free skeleton plus features covers almost everything worth checking, and the genuinely cross-serial constructions are rare enough to set aside.

5 From generation to constraints

The grammars above generate: start at S and rewrite down to w. The constraint-based frameworks (HPSG, LFG) invert this, a view called model-theoretic syntax (Pullum & Scholz, 2001): a grammar is not a device that produces strings but a set of constraints a structure must satisfy. Grammaticality becomes the existence of a structure for w satisfying every constraint.

A violated constraint is local — it concerns particular constituents — and named — it is a particular constraint. That is what makes error reporting possible at all, rather than a bare yes/no. The machinery is feature structures and unification.

Definition 5.1 (Feature structure)**.** A rooted, connected, directed acyclic graph with feature-labelled arcs and atomic leaf values, sharing of substructure allowed (reentrancy). Equivalently a partial function from feature paths to values, e.g. $[\,\mathsf{cat}\colon \mathsf{noun},\ \mathsf{num}\colon \mathsf{sg},\ \mathsf{pers}\colon 3,\ \mathsf{case}\colon \mathsf{nom}\,]$.

Definition 5.2 (Subsumption and unification)**.** $F \sqsubseteq G$ (F subsumes G) when G carries all of F's information and perhaps more — F is more general. The unification $F \sqcup G$ is the least upper bound in this order: the most general structure holding the information of both.

Theorem 5.1 (Unification is a lattice join)**.** *Under subsumption, feature structures form a lattice once a top element $\top$ (inconsistency) is adjoined. $F \sqcup G$ always exists and equals $\top$ exactly when F and G conflict on some shared path.*

Proof sketch. Compatible information merges to a unique least refinement, computed by a graph merge (union–find over the two DAGs, identifying nodes reached by the same path). Two paths demanding distinct atomic values — $\mathsf{num}\colon \mathsf{sg}$ against $\mathsf{num}\colon \mathsf{pl}$ — have no consistent refinement, so the join is $\top$: unification fails. □

Agreement is enforced by unifying the relevant constituents' feature structures. A subject with $\mathsf{num}\colon \mathsf{sg}$ against a verb with $\mathsf{num}\colon \mathsf{pl}$ unifies to $\top$: the parse fails, and the failure says which two constituents clashed and on which feature. That is the gap between a yes/no acceptor and one that can report a verb disagreeing with its singular subject. Kasper and Rounds (1986) gave this a logical semantics, so rules read as constraints rather than ad hoc checks; the verdict and the diagnosis come from one mechanism.

6 Gradience and the coverage trap

Two cracks in the purely formal program shaped the field afterward.

First, grammaticality is binary but acceptability is graded (the familiar "?", "??", "*"). This drove weighted grammars. A probabilistic context-free grammar attaches a probability to each production $A \to \beta$ with $\sum_\beta P(A \to \beta) = 1$; a tree's probability is the product of its productions, and probabilistic CKY returns the most likely tree. The field then moved to data-driven statistical parsing and, now, neural sequence models; modern grammatical error correction is sequence-to-sequence, not rule-based, because it tolerates gradience and gaps.

Second, and crucially, the coverage caveat is formal, not incidental. A grammar G defines $L(G)$, but any hand-built G is incomplete, so

$$\neg\big(w \in L(G)\big) \;\neq\; \text{“}w \text{ is ungrammatical.”}$$

No parse may mean only that G does not yet cover w. Treating "no parse" as "error" manufactures false positives. So a checker reads failure as *unrecognized*, not *ungrammatical*: it reports violated constraints it knows (with targeted error rules, or explicit mal-rules that match specific wrong patterns), never the bare absence of a parse.

7 Constraints on the rule design

What the theory dictates, collected:

- Anchor at the context-free level — membership must be decidable and polynomial (Theorem 3.1).
- Natural language exceeds context-free only mildly, on rare cross-serial constructions; defer them.
- Use feature structures and unification for agreement, government, and case; a unification failure already localizes and names the error.
- Scope the negative verdict honestly: report violated known constraints, never the absence of a parse.

8 The engine as a pipeline

Membership and diagnosis are computed in stages, each a function from one representation to the next:

```
text -> tokenizer -> morphology -> lexicon -> parser(+unification)
     -> error detection -> report
```

1. **Tokenizer.** Raw string → sequence of tokens.
2. **Morphology.** Each token → (lemma, morphological features), possibly several analyses.
3. **Lexicon.** Each analysis → a category and a feature structure.
4. **Parser.** Tokens → constituent structure over a context-free backbone, unifying features as it builds (Section 3 fixes this at the context-free level for decidability).

5. **Error detection.** Unification failures and matched error patterns → localized violations.

6. **Report.** Violations → highlighted span, named rule, suggested fix.

The pipeline is mostly language-independent; the lexicon, the morphology, and the rule set are the language-specific data it runs on. Stages 1–3 are where ambiguity enters — a token may have several analyses — and the parser resolves it by keeping only the analyses that fit into a structure.

9 Tokenization

The tokenizer realizes a map $\Sigma^* \to T^*$ from the symbol string to a token sequence over a vocabulary T. For space-delimited languages (English, Swedish) whitespace and punctuation carry most of the work, but the map is not trivial: contractions split (`don't` → `do` + `n't`), clitics and hyphenates need a policy, and punctuation attaches ambiguously. For languages written without spaces (Japanese) segmentation is the hard part — word boundaries are ambiguous and must be recovered by dictionary longest-match or a statistical model. Tokenization can itself be ambiguous, so it is better treated as a relation than a function, with the ambiguity passed downstream rather than resolved prematurely.

10 Morphological analysis

A word form decomposes into morphemes: a stem plus affixes. Inflectional morphology marks features — number, person, tense, case, gender — without changing category; derivational morphology changes it. The analyzer maps a surface form to one or more (lemma, features) pairs:

```
bark  ->  (bark, V, pres, agr != 3sg)     # "they bark"
bark  ->  (bark, N, num=sg)               # "a bark"
barks ->  (bark, V, pres, agr  = 3sg)     # "it barks"
barks ->  (bark, N, num=pl)               # "two barks"
```

Morphology is regular (Type 3 in Section 2), so finite-state transducers handle it — Koskenniemi's two-level model (Koskenniemi, 1983), which is fast and runs in both directions, analysis and generation. For English the morphology is light and the residual ambiguity (is `bark` a noun or a verb?) is left for the parser. For agglutinative (Japanese, Turkish) or fusional (Russian) languages this stage carries most of the grammatical information, and the feature structures it emits are correspondingly richer.

10.1 Transducers

Definition 10.1 (Finite-state transducer)**.** A finite-state transducer is a 5-tuple $T = (Q, \Sigma, q_0, F, \delta)$, with Q a finite state set, Σ a finite alphabet shared by both tapes, $q_0 \in Q$ the initial state, $F \subseteq Q$ the accepting states, and a transition relation

$$\delta \subseteq Q \times (\Sigma \cup \{\varepsilon\}) \times (\Sigma \cup \{\varepsilon\}) \times Q.$$

An arc $(p,\ x{:}y,\ q) \in \delta$ reads x on the input tape, emits y on the output tape, and moves from p to q; either symbol may be the empty string ε.

The relation $R(T) \subseteq \Sigma^* \times \Sigma^*$ is the set of pairs (u, v) such that a sequence of arcs spells u on the input tape, v on the output, and lands in F. Equivalently, $R(T)$ is the pair of input/output projections of a regular language over $\Sigma \times \Sigma$ — a *regular relation*. Type 3 on each projection, by definition.

Five operations carry the construction. *Union*, *concatenation*, and *Kleene closure* build $R \cup S$, $R \cdot S$, and R^* by the standard NFA constructions, with ε-arcs at the seams. *Inversion* swaps the tapes: T^{-1} has the same state graph with $x{:}y$ replaced by $y{:}x$ on every arc. *Composition* chains two transducers: if T_1 relates tapes X and Y and T_2 relates Y and Z, then $T_1 \circ T_2$ relates X and Z, with state space $Q_1 \times Q_2$ and an arc $(p_1, p_2) \xrightarrow{x:z} (q_1, q_2)$ whenever T_1 has $p_1 \xrightarrow{x:y} q_1$ and T_2 has $p_2 \xrightarrow{y:z} q_2$ for some y.

Theorem 10.1 (Closure)**.** *The regular relations are closed under union, concatenation, Kleene closure, composition, and inversion.*

Composition is the operation that lets a morphology be built out of pieces. *Application* in either direction is the same DFS through one relation, with inversion choosing which tape is consumed: for input u, $\mathsf{apply}_{\downarrow}(T, u) = \{v \mid (u, v) \in R(T)\}$ analyses, $\mathsf{apply}_{\uparrow}(T, v) = \{u \mid (u, v) \in R(T)\}$ generates, and $\mathsf{apply}_{\uparrow}(T, v) = \mathsf{apply}_{\downarrow}(T^{-1}, v)$.

10.2 Continuation lexicons

The lexicon part of the morphology — which stems take which suffixes, in what order, with what features — is itself a regular relation. The standard way to write it (Karttunen's LEXC) names blocks of stems and suffixes that reference each other by name. A `Root` block lists stems with their continuation classes; each continuation lists possible suffixes, each emitting a tag sequence on the lexical tape and consuming a (possibly empty) suffix string on the surface tape. So

```
%lex Root
    dog : N-reg  <lemma>=dog
%lex N-reg : N
    +N+Sg :       <num>=sg <pers>=3
    +N+Pl : s     <num>=pl <pers>=3
```

declares that from stem `dog` the paradigm `N-reg` produces two forms: lexical `dog+N+Sg` corresponds to surface `dog`, and lexical `dog+N+Pl` corresponds to surface `dogs`. Each block compiles to a sub-FST; cross-references between blocks are ε-arcs; the lexicon transducer is the union of all root-to-suffix paths.

Features attach to tags rather than to surfaces. A successful application returns a tag sequence, the decoder reads off which paradigm entries the sequence matched, and the root entry's features unify with the suffix entry's to yield the lexical analysis the parser consumes. Unification, not assignment: a singular-noun stem's `num:sg` flows into the lemma's record by the join of Section 5.

A real lexicon has stems that follow their paradigm almost perfectly but not quite: *fish* pluralises to itself, *go* takes *went* in the past. Inventing a single-stem paradigm for each is correct but inflates the paradigm namespace until it is no longer a paradigm namespace. The LEXC syntax extends one level: a root entry may be followed by *override entries*, indented one level deeper, each naming a paradigm tag whose surface and features it replaces on this root only. Override matching is by tag; an override naming a tag the paradigm does not carry is an error.

```
%lex Root
```

```
    fish : N-reg   <lemma>=fish
        +N+Pl : 0  <num>=pl

%lex N-reg : N
    +N+Sg :   <num>=sg <pers>=3
    +N+Pl : s <num>=pl <pers>=3
```

Formally, write the effective entry list for a root r in a paradigm p as

$$\mathcal{E}(r, p) \;=\; \big[\, \omega_r(e) \mid e \in p.\mathit{entries} \,\big],$$

where $\omega_r(e) = e$ if r does not override $e.\mathsf{tag}$, and otherwise $\omega_r(e)$ replaces the surface with the override's surface and merges the features by override-wins assignment at every path the override touches. Compilation and decoding both consume $\mathcal{E}(r, p)$ rather than $p.\mathit{entries}$ directly.

The naive translation — one sub-path per $(r, e) \in p.\mathit{entries}$ pair, the lot unioned — yields $\Theta(\text{stems} \cdot \text{forms} \cdot \text{stem-length})$ states. Fine for a 6-noun fragment, painful at realistic scale: ten thousand stems times ten forms each times a six-character mean stem length pushes well past a million states before composition. The construction in this engine groups roots by paradigm and, inside each group, builds a character trie over the stems: trie nodes are FST states, edges are identity arcs $c{:}c$. At each trie leaf an ε-arc enters a paradigm-level continuation sub-FST that emits the tags and consumes the suffixes for $p.\mathit{entries}$ once, shared by every root that follows p without overrides; the few roots with overrides each get a private continuation built from their $\mathcal{E}(r, p)$. The resulting state count is

$$O(\text{distinct stem prefixes}) \;+\; O(\text{paradigms} \cdot \text{forms}) \;+\; O(\text{overridden roots} \cdot \text{forms}),$$

an order of magnitude smaller in practice, and the relation is unchanged — apply over the trie plus shared continuation is identical to apply over the union of per-pair sub-paths.

10.3 Replace rules

A purely concatenative lexicon spells out every irregular surface explicitly, which is feasible for English and prohibitive for agglutinative or fusional languages. The standard alternative is a cascade of *replace rules* that mediate between an idealised lexical-side surface and the actual surface. Each rule has the form

$$A \to B \parallel L \_ R,$$

read: A on input rewrites to B on output, in the context where the left fragment L appears immediately before the position and R appears immediately after. Every such rule denotes a regular relation; the cascade composes them into a single relation, which then composes with the lexicon transducer.

Two semantics for the same syntax are classical (Karttunen, 1995):

- *Obligatory.* In context L_R, A *must* rewrite to B — the compiled FST has no path through the position that leaves A alone. Built by Karttunen's marker construction: insert brackets around every potential rewrite site, filter by context, replace within brackets, remove brackets. Five composed transducers, each delicate.
- *Optional.* In context L_R, A *may* rewrite to B, and identity is also permitted. The compiled FST has both paths. Built in one expression,

$$\Big(\; Id_1 \;\cup\; L \cdot \langle A : B \rangle \cdot R \;\Big)^{*},$$

where Id_1 is identity on a single Σ-symbol and $\langle A : B \rangle$ is the one-arc relation $\{(A, B)\}$.

Optional rules over-generate: a surface reachable only through "rewrite did not fire" is also accepted. This sounds wrong but is harmless once the lexicon transducer is composed downstream — the lexicon emits only well-formed lexical strings, so the spurious paths through the rule cascade fail to land on any lexical analysis and drop out of the final relation. The engine takes the optional path: a few lines, no marker machinery, and the lexicon is doing the work of filtering anyway.

10.4 An operational pitfall

There is one thing about applying composed transducers that is worth recording explicitly, because it is the kind of cost that is invisible at the level of the algebra and unmissable at the level of the runtime.

Composition produces an FST whose state count is at most $|Q_A| \cdot |Q_B|$ — polynomial. The number of ε-arcs, however, is not pinned by the state count. An ε-arc on either side of the composition seam survives into the result, because the machine on the other side of the seam does not advance when its partner emits or reads ε. A few rule transducers built by the optional-replace construction are state-sparse but ε-dense (every Kleene loop contributes its own ε-back-edge, and every union arm an ε-front-edge), and their composition is more so.

Application by depth-first search through such a graph is exponential. The DFS at each $(\mathsf{state}, \mathsf{position})$ tries every outgoing arc, including ε-arcs that re-enter states without consuming input. A path-local visited set — $(\mathsf{state}, \mathsf{position})$ marked on entry, unmarked on backtrack — ensures termination, but does not prune across paths: each distinct path that reaches a given $(\mathsf{state}, \mathsf{position})$ is searched afresh, and in a dense ε-graph the number of such paths is exponential in depth.

We confirmed the cause by bisection. Constructing each rule transducer took microseconds (tens of states each). Composing two of them produced $\sim$ 3800 states in milliseconds — fast, polynomial as predicted. Applying the composed machine to a single input character hung past thirty seconds. The state count was fine; the path count was not.

Remark. The principled fix is the marker construction for obligatory replace, which yields a nearly deterministic FST with bounded ε-fanout. That is a real piece of work and is postponed.

The pragmatic fix gives up the single composed transducer. The rule cascade is kept as a list $\langle T_1, T_2, \ldots, T_k \rangle$ and applied sequentially: for input x,

$$\mathsf{apply}_{\downarrow}(T_k, \mathsf{apply}_{\downarrow}(T_{k-1}, \ldots \mathsf{apply}_{\downarrow}(T_1, \{x\}))),$$

with deduplication between layers. Composition is associative, so the set of outputs equals the set of outputs of the composed relation; nothing is lost mathematically. The fan-out per layer is bounded by the number of optional rewrite sites in the layer's input, which is at most that input's length; the total fan-out is the product across layers, finite and small for any realistic cascade. We pay one application per rule rather than one application on the composed machine, and that constant factor is what buys the polynomial runtime back.

Composition stays in the kernel as a primitive — the lexicon needs it to join continuation blocks, where the ε-density is mild — but the rule cascade does not use it.

The pragmatic fix solves the runtime problem without addressing the structural one. There is a second, more local, principled fix that targets the apply-time pathology at its root: *input-ε elimination*. The transformation is the transducer analogue of the standard ε-elimination for finite automata. For each state s, compute the input-ε closure

$$E_{\varepsilon}^{\downarrow}(s) \;=\; \left\{ (t,\, w) \;\middle|\; s \xrightarrow{\varepsilon:w_1} \cdots \xrightarrow{\varepsilon:w_n} t,\;\; w = w_1 \cdots w_n \right\},$$

the set of (t, w) pairs reachable from s by an input-ε path with concatenated output w. For every non-ε-input arc $t \xrightarrow{x:y} u$ leaving any state t in the closure, install a new arc $s \xrightarrow{x:wy} u$ in the rebuilt transducer. If $(t, w) \in E_{\varepsilon}^{\downarrow}(s)$ has $t \in F$, then s becomes an accepting state whose acceptance emits w on the output tape as exit output. Output-ε arcs are left in place; they are harmless, since termination does not depend on them.

The rebuilt transducer accepts exactly the same relation as the original. The proof is the usual closure argument: every accepting run in the original factors as a finite alternation of input-ε segments and non-ε-input arcs, and each non-ε arc is the witness for exactly one new arc in the rebuilt machine (with the preceding input-ε segment folded into its output); conversely every run in the rebuilt machine corresponds to a unique factored run in the original. Acceptance via exit output covers the residual case in which the last segment is input-ε.

The payoff is that walking k arcs in the rebuilt FST consumes exactly k input symbols. The DFS depth is bounded by $|input|$ and the path-local visited set is no longer load-bearing — its only remaining duty is to guard against output-ε cycles, which the constructions in this engine do not contain. The cascade-blowup probe that hung past thirty seconds on length-one input completes in milliseconds after elimination, and composition becomes a usable operation again. Two implementation points are worth recording. First, arc outputs may be multi-symbol strings after elimination (since the closure concatenates several output symbols into the prepended prefix w), and apply re-tokenises the accumulated output at the end so callers always see a one-symbol-per-element sequence. Second, exit outputs are stored as a per-state list rather than a single string, since one state may ε-reach distinct finals along distinct outputs — the override-only *fish* case in Section 10.2 is the smallest example. Inversion, used by $\mathsf{apply}_{\uparrow}$, materialises each exit-output string as a chain of input arcs ending at a fresh accepting state, so generation keeps working through the same kernel.

Remark. Marker construction and input-ε elimination address different problems. Marker construction restores obligatory replace semantics and pins ε-fanout as a consequence; input-ε elimination takes the existing ε-dense optional-replace transducers as given and makes their apply tractable. The engine uses elimination unconditionally; obligatory replace remains a future task, taken on the day a rule actually requires it.

10.5 Pipeline placement

The compiled morphology slots into the position Section 8 reserves for it. The interface is a function from a surface token to a (possibly empty) list of $(\mathsf{lemma}, \mathsf{cat}, \mathsf{feats}, \mathsf{err}?)$ analyses, looked up in this order:

1. explicit lexicon entry (closed-class items, irregulars);
2. paradigm-derived analysis via the FST;
3. empty ⇒ unknown word.

The closed-class lexicon and the FST jointly partition the open vocabulary; the explicit lexicon doubles as the override mechanism for items that have no paradigm to follow (`the`, pronouns, prepositions) or that are genuinely irregular. Regular inflection — including the past tense, once the `%rule` cascade of Section 10.7 is in place — is paradigm-derived; `chased` is no longer an explicit entry. Ambiguity propagates as a list — a form that analyses as both noun and verb produces both entries — and the parser resolves which fit into a tree. The optional err field, populated only for error-tagged paradigm entries, is the subject of the next subsection.

10.6 Morphological mal-rules

Anticipated *wrong* surfaces (`*childs`, `*goed`, `*mans`) need a place to land. Treating them as unknown words is the wrong report: the user knows what they meant; they wrote the wrong plural. Phase 8 of the engine closes the loop by giving the morphology a counterpart of the syntactic mal-rules of Section 11. A paradigm entry may carry an `*ERR "..."` message and a `*FIX <form>` suggestion; the entry describes an anticipated wrong surface, and the lexicon FST contains it alongside the canonical entry under the same tag.

```
%lex N-irreg-children : N
    +N+Sg :     <num>=sg <pers>=3 <vagr>=3sg
    +N+Pl : ren <num>=pl <pers>=3 <vagr>=n3sg
    +N+Pl : s   <num>=pl <pers>=3 <vagr>=n3sg
                *ERR "irregular plural: 'children', not '*childs'"
                *FIX children
```

Both `+N+Pl` entries share a tag. The FST emits `child+N+Pl` for either surface; decode_lexical returns both candidate analyses (different surfaces, same tag sequence); and morph_analyze narrows by the actual surface form, attaching the error to the wrong-surface analysis and leaving the right-surface analysis clean. The decode step violates an invariant that held silently through Phase 7 — that the lexical tape uniquely identified the (root, entry) pair — and the surface-form filter is the price of relaxing it.

The error then flows through the engine as a leaf-level violation. The chart scanner builds a leaf edge whose `violations` list contains {rule:"morph", message: ..., fix_strategies:["literal:children″], span:$[i, i{+}1]$}, and advance propagates it through the chart unchanged. The min-violations packing of Section 13 prefers any clean leaf at the same position, so a clean and an error-tagged analysis for the same surface would never both appear in the winning derivation.

The verdict cascade collapses what would otherwise be a new branch. The model now distinguishes three states for a surface form: *clean* (paradigm- or lexicon-derived without error), *error-tagged* (a known malformation with a suggested fix), and *unknown* (no analysis at all). The first two are admitted to the parser; the third skips it. A parse with leaf-level violations is reported as `ungrammatical` with those violations attached, indistinguishable in verdict shape from a syntactic mal-rule. The downstream report can colour-code by violation.rule if it cares about the distinction; the analyzer does not pre-bin. The coverage caveat of Section 6 holds: the model only reports a malformation when it has explicitly anticipated it. A surface for which no error-tagged path exists falls through to unknown, exactly as before.

Remark. The mal-rule mechanism here, like `%mal` for syntax, is an admission that the model is incomplete: every error type the engine recognises has to be authored by hand. The notes do not pretend otherwise. What the mechanism buys is the right place to put the next admitted error — author it once in the grammar file; the rest of the pipeline already knows how to carry it.

10.7 Productive paradigms and the analysis direction

For several phases the replace-rule cascade of Section 10.3 was machinery without a caller: the rule compiler, the apply primitives, and input-ε elimination all existed, but no paradigm referred to a rule and the lexicon spelled out every surface, past-tense `chased` included. The `%rule` directive (Section 15) supplies the caller. A paradigm entry may now store an *underlying* suffix, and a cascade realises the orthographic surface from it. English regular past tense is the motivating case: the single entry and two rules

```
%lex V-reg : V
    +V+Past : ed  <tense>=past
%rule e:0 => Cons _ e d      # chase + ed -> chased   (drop a silent e)
%rule y:i => _ e d           # try   + ed -> tried    (y becomes i)
```

replace the per-verb explicit entries; `chased`, `liked`, `barked`, `tried` are all paradigm-derived.

Three representations, two relations. The lexicon transducer's lower tape, until now read as the final surface, is reinterpreted as a *morphophonemic* string — stem followed by the stored underlying suffix, e.g. `chaseed` for `chase` + `ed`. Write U for the morphophonemic strings, T for the tag strings, S for the orthographic surfaces. The lexicon is a regular relation $\mathrm{Lex} \subseteq U \times T$ with $\mathsf{apply}_{\downarrow}(\mathrm{Lex}, u) = \{\, t \mid (u,t) \in \mathrm{Lex} \,\}$; the cascade is a regular relation $\mathrm{Cas} \subseteq U \times S$ written in the generation direction, so $\mathsf{apply}_{\downarrow}(\mathrm{Cas}, u)$ realises surfaces and $\mathsf{apply}_{\uparrow}(\mathrm{Cas}, w) = \{\, u \mid (u,w) \in \mathrm{Cas} \,\}$ recovers morphophonemic pre-images.

Proposition 10.2 (Analysis is the composed pre-image)**.** *The analyses of a surface w are*

$$\mathcal{A}(w) \;=\; \{\, t \in T \mid \exists\, u \in U :\; (u,w) \in \mathrm{Cas} \;\wedge\; (u,t) \in \mathrm{Lex} \,\} \;=\; \pi_T\big((\mathrm{Cas}^{-1} \circ \mathrm{Lex})[\,w\,]\big),$$

and may be computed without ever composing the two machines, as

$$\mathcal{A}(w) \;=\; \bigcup_{u \,\in\, \mathsf{apply}_{\uparrow}(\mathrm{Cas},\, w)} \mathsf{apply}_{\downarrow}(\mathrm{Lex},\, u).$$

This is exactly the engine's analysis loop: run the cascade *upward* on the surface to get a finite candidate set of morphophonemic strings, then run each *downward* through the lexicon to tags. The avoidance of an explicit composition is the lesson of Section 10.4 re-applied — we never materialise $\mathrm{Cas}^{-1} \circ \mathrm{Lex}$, only its action on one input. With an empty cascade ($\mathrm{Cas} = \mathit{Id}$) the candidate set is $\{w\}$ and the loop degenerates to the old direct lookup, so grammars with no `%rule` behave bit-for-bit as before.

No boundary symbol. We deliberately omit the morpheme-boundary diacritic of the textbook construction. There, the stem/suffix seam is marked (`^`) so a rule can name "immediately before a suffix" exactly and a final rule deletes the marker, letting `e:0` fire only at a true junction. The engine instead lets rule contexts mention the suffix letters directly (`e:0 => Cons _ e d`) and leans on the lexicon-as-filter argument of Section 10.3: context-only rules over-generate, but the over-generation is discarded downstream because no lexical analysis lands on a spurious string. The price of skipping the boundary is a single *under-rejection*: because Cas contains identity, the raw morphophonemic form `chaseed` is itself a candidate that the lexicon accepts, so the analyzer admits it as a (non-word) spelling of `chase+V+Past`. This only ever fails to flag a non-word; it never mis-flags a real one, and it keeps intact the surface-literal contract that the lexicon FST and its round-trip generation tests were built on. A boundary symbol is the principled fix, postponed on the same terms as the marker construction.

One invariant shifts. Decoding a tag sequence back to a (root, entry) pair (Section 10.6) used to narrow tag-shared candidates (`*childs`) by the input *surface*. Once rules alter the surface, root.surface + entry.surface is the morphophonemic candidate u, not the input w; the narrowing is therefore by u — the lexicon path that actually matched — while the *reported* form is the true input w. The `*childs` disambiguation survives unchanged: the entry that reconstructs to the matched candidate is kept (`ren` for the candidate `children`, the error-tagged `s` for the candidate `childs`), and the error rides the wrong-surface analysis exactly as before.

11 Feature structures in the grammar

Section 5 gives feature structures and unification abstractly; in the engine they attach to two things. Lexicon entries carry one feature structure each. Grammar rules carry *equations* over the features of their constituents — the constraints. A small English fragment:

```
S  -> NP VP    : NP.agr = VP.agr      # subject-verb agreement
NP -> Det N    : NP.agr = N.agr
VP -> V        : VP.agr = V.agr
Det -> the
N   -> dog     : [num=sg, pers=3]
V   -> bark    : [agr != 3sg]
V   -> barks   : [agr  = 3sg]
```

A context-free backbone annotated with feature equations is a *unification grammar*; PATR-II is the minimal such formalism. As the parser applies a rule it unifies the daughters' structures according to the rule's equations and derives the mother's. A satisfied equation propagates information upward; a violated one yields $\top$ and pins the failure to a specific rule and a specific pair of constituents. The verdict and its explanation are then the same computation seen two ways.

12 Mal-rules and targeted error recognition

The coverage trap (Section 6) forbids reading "no parse" as "ungrammatical." Two strategies turn an absent parse into a specific, reportable diagnosis.

Constraint relaxation. When unification fails, retry the parse with one constraint dropped. If relaxing exactly the agreement equation `NP.agr = VP.agr` lets the parse complete, that equation is the diagnosis and the span is the constituents it related. General, but it over-triggers when several relaxations succeed at once.

Mal-rules. Encode a known wrong pattern as a rule carrying an error label, an explanation, and a correction. From the language-learning literature, mal-rules target specific recurrent errors:

```
S -> NP VP  : NP.agr = 3sg, VP.agr != 3sg
            *ERR  subject-verb agreement (missing 3sg -s)
             FIX  inflect verb to 3sg, or pluralize subject
```

Precise but anticipatory — each error must be foreseen and written. A working engine parses normally first and only on failure falls back to relaxation and mal-rules, ranked by likelihood, reporting the *minimal* set of violated constraints rather than every possible complaint.

13 A worked example: "the dog bark"

Trace the pipeline on the ungrammatical `the dog bark`.

Tokenize. `[the, dog, bark]`.

Morphology and lexicon. `the` is a determiner, unmarked for number. `dog` is a noun, `[num=sg, pers=3]`. `bark` is ambiguous: a present-tense verb requiring `agr != 3sg`, or a singular noun.

Parse. Reading `bark` as a verb:

```
       S
      / \
    NP   VP
   /  \   |
Det   N  V
the  dog bark
```

The subject NP inherits the noun's features, `NP.agr = [num=sg, pers=3]` (that is, 3sg). The VP inherits the verb's requirement, `VP.agr = [agr != 3sg]`. The rule `S -> NP VP` demands `NP.agr = VP.agr`. For a third-person subject the base form's requirement reduces to $\mathsf{num}:\mathsf{pl}$, so the parser unifies the subject's number with the verb's:

$$[\,\mathsf{num}:\mathsf{sg}\,] \sqcup [\,\mathsf{num}:\mathsf{pl}\,] = \top.$$

The values conflict, unification fails, and no S is built. Reading `bark` as a noun gives `Det N N`, which no rule licenses as a sentence, so that branch dies too. The sentence has no parse — but it failed on exactly one relaxable constraint, which the matching mal-rule names.

Report.

```
the dog bark
        ^^^^   subject-verb agreement error

Verdict:   ungrammatical

Violation: subject-verb agreement (number)
           subject "the dog" is 3rd person singular;
           "bark" is the non-3sg present form, which needs a
           non-3sg subject. Unifying num=sg with the required
           num=pl fails.

Rule:      S -> NP VP   with   NP.agr = VP.agr

Fix:       "the dog barks"   (inflect the verb for 3sg), or
           "the dogs bark"   (make the subject plural)
```

This is the target behaviour: a verdict, the offending span, the violated rule, and a concrete repair — all falling out of a single failed unification rather than a hand-written special case.

14 Auxiliaries, clitics, and the symbol-keyed environment

Completing a rule $A \to B_1 \cdots B_n$, the parser builds an environment ρ mapping each constituent's category symbol to a feature structure: it seeds $\rho(A)$ with the empty structure, then sets $\rho(B_i)$ to daughter i's structure, unifies the rule's equations against ρ, and reads the mother's structure back from $\rho(A)$. Keying ρ by *symbol* rather than by position has a deliberate consequence. When the mother shares a daughter's category — $A = B_i$ for some i — the assignment $\rho(B_i) \leftarrow$ daughter overwrites $\rho(A)$, so the mother *is* that daughter. For `NP -> NP PP` and `VP -> VP PP` this is precisely head inheritance, obtained for free: the mother takes the head daughter's features with no equation written. The repeated-category caveat noted at the end of Section 18 is the same coin's other face — a rule with two same-category daughters cannot name them apart.

That free inheritance is a hazard the moment the mother should *not* be the like-named daughter. English auxiliaries are the case in point. The natural rule `VP -> Aux VP` makes the mother `VP` inherit the inner (lexical verb) `VP`, so a finite auxiliary cannot impose its own agreement: under that rule `he'll bark` would inherit `bark`'s [vagr:n3sg] and clash with the third-singular subject `he`, mis-rejecting a grammatical sentence. The fix is to project the auxiliary through a fresh nonterminal, breaking the symbol collision:

```
AuxP -> Aux VP        : AuxP.vagr = Aux.vagr
AuxP -> Aux Neg VP    : AuxP.vagr = Aux.vagr
VP   -> AuxP          : VP.vagr   = AuxP.vagr
```

Because `AuxP` $\neq$ `VP`, no environment entry is overwritten; agreement is read off the auxiliary head, and the unit rule `VP -> AuxP` lifts the projection back to a `VP` so the sentence rule `S -> NP VP` still applies. A modal like `'ll` carries no `vagr`, leaving `AuxP.vagr` unconstrained, so it unifies with any subject (`he'll bark` and `they'll bark` both parse); a number-marked auxiliary like `'ve` carries [vagr:n3sg] and clashes with a third-singular subject (`he've` is rejected). The general principle: *a recursive same-category rule whose mother should not take the like-named daughter's features must introduce a fresh nonterminal*; the symbol-keyed environment turns category identity into structure identity, which is exactly what head projection wants and exactly what auxiliary projection must avoid.

The lexical material is supplied by the clitics the tokenizer splits off (Section 9). `don't` becomes `do` + `n't`, `I'm` becomes `I` + `'m`, and so on; the split pieces are unknown words unless the lexicon names them. Seven clitic forms (`n't`, `'m`, `'re`, `'s`, `'ve`, `'ll`, `'d`) join the closed-class lexicon, alongside the full copula and do-support forms (`is`, `am`, `are`, `do`, `does`) needed because `isn't` and `doesn't` split to those stems plus `n't`. Two clitics are genuinely ambiguous — `'s` is *is* or *has*, `'d` is *would* or *had* — and get one lexicon entry per reading; the parser keeps whichever yields a parse, the same list-valued ambiguity the morphology already produces. Negation and do-support fall out of `AuxP -> Aux Neg VP`. One incompleteness is admitted in the same spirit as the coverage caveat of Section 6: the morphology draws no finite/nonfinite distinction, so `I'll barked` (modal followed by a finite past) is not rejected. That is a false negative — a missed error, never a wrongly flagged correct sentence — and is left until a verb-form feature exists to state the constraint.

15 The declarative front end

The `.gram` file began as lexicon entries, phrase-structure rules, and mal-rules. As the grammar grew it accreted a layer of *directives* — lines beginning `%` — that do not describe constituents directly but configure the pipeline that processes them. The analogy is a compiler's preprocessing and declaration layer: `%include` is textual inclusion, `%feature` is a type declaration, `%class` is a macro over symbol sets, and `%rule` loads the morphological back end. They share the property that all of their work is done before any sentence is analysed.

15.1 `%feature`: a typing discipline on values

Feature values are atoms, and unification of distinct atoms yields $\top$ (Section 5). A misspelt value is therefore not an error but a silent non-event: writing `<num>=sgular` produces a well-formed atom that happens to unify with nothing the grammar ever asserts, so the entry simply never participates in a successful parse, and the symptom is a missing analysis far from its cause. The `%feature` directive turns this class of bug into a load-time error by declaring, for each feature,

the finite set of atoms it may take (or `*` for an open feature such as `lemma`, whose values are unbounded). The declaration induces a typing judgment: with $\mathsf{dom}(f) = \{v_1, \ldots, v_k\}$ declared, an occurrence $f = v$ is well-typed iff $v \in \mathsf{dom}(f)$, and an undeclared feature is ill-typed outright. A validator walks every feature structure the grammar produces — lexicon entries, the value side of rule equations, and paradigm features — and rejects the first ill-typed atom with its location. The check is opt-in: a grammar that declares no features is not validated, so the discipline is available without being imposed. Formally it is a soundness filter, not a completeness one — it cannot certify that a feature is *used* correctly, only that every value mentioned is one the feature is declared to range over — but that single guarantee converts the most common authoring mistake from a silent semantic failure into a named, located, load-time rejection.

15.2 `%class`: regular abstraction over rule contexts

Replace-rule contexts (Section 10.3) frequently want to name a phonological class — "after a consonant," "before a vowel" — rather than a single symbol. `%class Vowel : a | e | i | o | u` binds a name to a finite set of symbols. In a rule context the name denotes the single-symbol identity over that set,

$$Id_{\mathsf{Vowel}} \;=\; \bigcup_{c \,\in\, \mathsf{Vowel}} \{\,(c, c)\,\},$$

a regular relation, spliced into the rule transducer exactly where a literal symbol's identity would sit. A rule such as `e:0 => Cons _ e d` thereby deletes a stem-final `e` only after a consonant, without enumerating the twenty-one consonants by hand. One parsing subtlety: a class name must survive tokenisation as a single opaque symbol — `Cons` must not be read as `c o n s` — so the class table is threaded into the rule parser, which consults it before falling back to character-level tokenisation.

15.3 `%include` and `%import`: modularity and the load/run split

As the closed-class lexicon, the open-class morphology, and the syntax grew, a single file stopped being the right unit. `%include` splices another `.gram` file's lines in place; `%import` reads a tab-separated lexicon and expands each row into a `%lex Root` entry. Both are a source-to-source pass run before parsing: a function

$$\mathsf{Expand} : \Sigma^* \times (\mathsf{path} \to \Sigma^*) \;\to\; \Sigma^*,$$

taking the root text and a *resolver* — a map from a relative path to its contents — to a single flat line stream with every directive discharged. Expansion is recursive and the include graph must be acyclic; a back-edge is a load error, detected by a visited-set on the resolution stack. Each emitted line carries an origin (file and line number) so that a parse error downstream points into the source the author actually wrote, not into the flattened stream.

Injecting the resolver rather than reading files directly is what lets the same parser run in two environments with opposite constraints. Under Node the resolver reads from disk. In the browser there is no filesystem at run time, so the expansion is performed once at *build* time — the grammar is flattened to a single directive-free string and bundled — and the run-time parser is handed text with no directives left to resolve. This is the load-time/run-time separation a compiler makes between preprocessing and execution, here doubling as the portability seam between a server and a browser: directive resolution is a build concern, analysis is a run concern, and the resolver is the one interface between them.

16 Accidental complexity in decoding and fix generation

Section 10.4 recorded a cost invisible in the algebra and unmissable at the runtime. Two more of the same shape surfaced under a load the unit grammar never exercised — a ten-thousand-word imported lexicon — and both are worth recording, because both are the same mistake: an operation that is silently linear in the size of the lexicon, sitting on a path that the diagnosis machinery re-runs many times.

Decoding. The decoder of Section 10.6 mapped a lexical tape back to its $(\mathsf{root}, \mathsf{entry})$ analyses by scanning *every* root and testing whether its surface is a prefix of the tape — $O(R)$ per analysed token, with R the number of roots. The lexical tape is, by construction, the stem's characters (one symbol each) followed by the tag symbols, so the stem is recoverable by its content alone. Indexing the roots by surface, $\mathsf{idx} : \Sigma^* \to \mathcal{P}(\mathsf{Root})$, and walking the prefixes of the tape — testing each prefix against idx — finds the stem in $O(\ell)$ lookups, ℓ the stem length, independent of R. The walk preserves the scan's exact semantics: overlapping stems (`do` and `dog`) are both found, at their respective prefix lengths, and the prefix is built by concatenating tape *symbols* rather than by indexing into a string, so multi-code-unit characters stay consistent with how the trie emitted them. The index is memoised on the (post-parse immutable) morphology, so the $O(R)$ construction is paid once per grammar rather than once per token.

Fix generation. When a parse fails on a relaxable constraint, the engine proposes repairs (Section 12): for `agree-verb` it substitutes another form of the same verb, for `add-determiner` it inserts each determiner, and so on. Each strategy drew its candidates from a helper that materialised the *entire* surface inventory — explicit entries plus every $(\mathsf{root} \times \mathsf{entry})$ expansion, with a unification apiece — on every call, $O(L)$ for $L \approx R \times$ forms. Several strategies fire per diagnosis, and each candidate is re-parsed to confirm it lowers the violation count, so the linear cost was multiplied by candidate count and by re-analysis. The fix is the same shape as for decoding: build the inventory once per grammar, memoised, and index it by the keys the lookups actually use — by category, and by $(\mathsf{category}, \mathsf{lemma})$ — so each lookup is $O(\mathsf{matches})$ rather than $O(L)$.

The effect, measured on the ten-thousand-word lexicon over a mixed workload of grammatical, mal-rule, and relaxed-diagnostic sentences:

input class (median latency)	original	+ root index	+ fix index
grammatical	4.64 ms	1.29 ms	1.43 ms
mal-rule (word order)	8.94 ms	2.63 ms	2.59 ms
relaxed (subject–verb agreement)	44.3 ms	25.8 ms	7.22 ms
relaxed (missing determiner)	75.1 ms	33.4 ms	15.5 ms
overall p95	77 ms	34 ms	16.7 ms

The proof point is that the relaxed paths, after both indexes, run at the same latency on the ten-thousand-word grammar as on the two-word grammar — analysis and repair are now independent of lexicon size. What remains in the relaxed rows is the candidate-count multiplier of re-parsing repairs, which scales with the grammar's rule set and the number of fix strategies, not with the lexicon, and is the irreducible cost of validating suggestions rather than an accidental complexity to be removed.

Remark. The general lesson, stated once: an operation that is linear in the lexicon is invisible on a toy grammar and dominant at scale, and the diagnosis path amplifies it by re-running analysis on every repair candidate. The remedy in both cases was to precompute an index keyed by the lookup actually performed and to memoise it on the grammar, which is immutable once parsed.

The benchmark, not the algebra, is what made the cost visible — the same bisection discipline as Section 10.4.

17 Explainability and the chart trace

The engine's premise is that a verdict should come with its reason, and the same commitment extends to the engine's own behaviour during development. A trace mode narrates the analysis to a side channel without altering the verdict: it prints each token's morphological analyses, then each step of the chart parse — PREDICT, SCAN, COMPLETE — flagging where a prediction finds no rule, where a scan finds no token of the required category, and where a completion is blocked by a feature clash, before dumping the final chart by column. The dead-ends are the useful part: an absent parse (Section 6) is silent by nature, and the trace turns it into a located event — "at column i, expected a `V`, found only an `Aux`" — which is the difference between a coverage gap one can see and one one cannot. The same explainability the engine offers its user, it offers its author.

18 The grammar file format

The engine reads a grammar from a plain-text `.gram` file. The format is a concrete syntax for the unification grammar of Section 11 — PATR-II in spirit (Shieber, 1986) — with the diagnostic and mal-rule lines of Section 12 added. There are three statement kinds: lexicon entries, phrase-structure rules, and mal-rules. Whitespace separates tokens but is otherwise insignificant; `#` begins a comment running to end of line; blank lines are ignored. The grammar of the format (`::=` defines, `|` alternates, `{}` repeats zero or more, `[]` is optional, `EOL` is end of line):

```
grammar    ::= { lexentry | rule | malrule | directive }

lexentry   ::= word ":" cat { featpath "=" value } EOL

rule       ::= production { equation }
production ::= sym "->" sym { sym } EOL
equation   ::= rulepath "=" ( rulepath | value ) [ diag ] EOL
diag       ::= "!" string [ "fix:" strategy { "|" strategy } ]

malrule    ::= "%mal" production "*ERR" string [ "*FIX" text ]

directive  ::= feature | class | replrule | include | import
feature    ::= "%feature" feature ":" ( value { "|" value } | "*" ) EOL
class      ::= "%class" name ":" sym { "|" sym } EOL
replrule   ::= "%rule" pat ":" pat [ "=>" { ctxsym } "_" { ctxsym } ] EOL
include    ::= "%include" path EOL          # splice another .gram file
import     ::= "%import"  path EOL          # TSV: stem TAB paradigm TAB feats
ctxsym     ::= sym | name                   # a literal symbol or a %class

featpath   ::= "<" feature { feature } ">"        # implicit constituent: the
                                                  # entry's own structure
rulepath   ::= "<" sym { feature } ">"            # first sym = a constituent
                                                  # of the production; the rest
```

```
                                                    # navigate into its structure

word       ::= token            # surface form, e.g. dog, barks
cat, sym   ::= identifier       # category / nonterminal: N NP V Det Pron S
feature    ::= identifier       # num pers case vagr
value      ::= identifier       # sg pl 3 nom acc 3sg n3sg
strategy   ::= identifier       # agree-verb nominative-pronoun ...
string     ::= '"' { char } '"'
```

The semantics the syntax does not show:

- `=` is unification, not assignment (Section 5): symmetric, order-independent, a declared constraint rather than a store. All of a rule's equations are unified together when the rule applies; any conflict fails the application and, with it, the parse along that branch.
- A `rulepath` names a constituent by its category symbol, then a feature sequence into that constituent's structure. In a `lexentry` the constituent is implicit — the entry's own structure — so the path is features only.
- An omitted feature is unconstrained and unifies with anything; grammatical facts such as "the past tense marks no agreement" are stated by saying nothing (Section 11).
- A `!`-tagged equation carries the diagnostic reported when relaxing exactly that constraint rescues the parse; `fix:` names repair strategies. A `%mal` rule instead recognizes a wrong pattern directly and always reports.
- Directive lines (`%`-prefixed) are discharged before parsing: `%include` and `%import` expand textually (Section 15), `%feature` declares the value types checked at load, `%class` names a symbol set usable in rule contexts, and `%rule` adds a morphophonemic replace rule to the cascade (Section 10.7).

One caveat in the present design: a path identifies a constituent by category symbol, which is unambiguous only when that symbol occurs once in the production. A rule with a repeated category (`S -> NP NP`) would need indexed names (`NP.1`, `NP.2`); the current fragment avoids the situation. The same symbol-keying is what gives free head-feature inheritance and what forces the auxiliary-projection workaround of Section 14.

References

[1] N. Chomsky. *Three models for the description of language.* IRE Transactions on Information Theory, 1956.

[2] S. M. Shieber. *Evidence against the context-freeness of natural language.* Linguistics and Philosophy, 1985.

[3] A. K. Joshi. *Tree adjoining grammars: How much context-sensitivity is required to provide reasonable structural descriptions?* In Natural Language Parsing, 1985.

[4] K. Vijay-Shanker and D. J. Weir. *The equivalence of four extensions of context-free grammars.* Mathematical Systems Theory, 1994.

[5] G. K. Pullum and B. C. Scholz. *On the distinction between model-theoretic and generative-enumerative syntactic frameworks*. In LACL, 2001.

[6] R. Kasper and W. Rounds. *A logical semantics for feature structures*. In ACL, 1986.

[7] K. Koskenniemi. *Two-Level Morphology: A General Computational Model for Word-Form Recognition and Production*. Publications of the Department of General Linguistics, University of Helsinki, 1983.

[8] L. Karttunen. *The replace operator*. In ACL, 1995.

[9] D. Jurafsky and J. H. Martin. *Speech and Language Processing*, 3rd ed. (draft).

[10] M. Sipser. *Introduction to the Theory of Computation*.

[11] S. M. Shieber. *An Introduction to Unification-Based Approaches to Grammar*. CSLI, 1986.

[12] F. C. N. Pereira and S. M. Shieber. *Prolog and Natural Language Analysis*. CSLI, 1987.